

AI-Augmented Backend Development Project

Project Title: **AI Blog Nest API – An Intelligent Backend for Blog Operations**

Team ID : SWTID-2026-4636

Team Leader : SAI TEJA SHRI P

Team member :

SAI TEJA SHRI P

YAMINISREE P

HARSHITHA K

ISHWARYALAKSHMI S

Project Description:

AI Blog Nest API is an AI-Augmented backend web application developed using **Node.js, Express.js, and MongoDB**, integrated with Artificial Intelligence capabilities for intelligent blog content management. The system provides secure user authentication, blog creation and management, AI-powered content generation, text summarization, and intelligent content processing through AI APIs. Users can register, log in, create and manage blog posts, generate blog content based on a topic and category, and summarize existing content using AI. By combining traditional backend development with modern AI capabilities, AI Blog Nest provides an efficient, scalable, and user-friendly platform for creating, managing, and processing blog content.

Scenario:

In the modern digital environment, creating and managing high-quality blog content can be time-consuming. Bloggers, students, developers, and content creators need to research topics, organize information, write articles, and summarize lengthy content manually. Traditional blog management systems mainly provide CRUD operations for creating, reading, updating, and deleting posts, but they do not provide intelligent assistance for generating or processing content. This increases the time and effort required to maintain useful and engaging blog content.

Artificial Intelligence can improve this process by assisting users with content generation and text processing. **AI Blog Nest API** integrates AI services into the backend to generate blog content based on a given topic and category and summarize existing content. The application uses REST APIs to communicate with the AI service and returns structured responses that can be consumed by a frontend application or tested using Postman.

The system benefits bloggers, students, developers, and content creators by reducing manual content creation effort and providing faster access to AI-assisted writing and summarization. At the same time, JWT authentication and protected APIs help secure user data and blog operations. The project demonstrates how conventional backend technologies can be combined with Artificial Intelligence to create an intelligent content-management platform.

System Requirements

To ensure smooth development, deployment, and usage of the **AI Blog Nest API**, the following system prerequisites are required.

1. Software Requirements

Operating System

- Windows 10/11

Node.js

- Node.js v18 or above

Provides the runtime environment required to execute the backend application.

npm

- **npm v9 or above**

Used for installing and managing project dependencies.

Express.js

Backend framework used for:

- REST API development
- Routing
- Middleware
- Request and response handling

MongoDB

MongoDB is used as the database for storing:

- User information
- Blog posts
- Categories
- AI-generated content
- Application data

MongoDB Atlas

MongoDB Atlas can be used as the cloud-hosted MongoDB database for the application.

AI Service Provider

The application can integrate an AI API such as:

- OpenAI API
- Google Gemini API
- Claude API
- Azure OpenAI

OpenAI API is used in the project for AI-powered blog content generation and summarization.

JWT

JSON Web Token is used for:

- User authentication
- Protected API access

- Token-based authorization

Mongoose

Mongoose is used for:

- MongoDB connection
- Schema definition
- Data validation
- Database operations

Postman

Used for testing and demonstrating:

- Authentication APIs
- Blog APIs
- AI APIs
- Protected routes
- Request and response structures

Visual Studio Code

Development environment used to create and manage the backend project.

Git & GitHub

Used for:

- Source-code management
- Version control
- Project collaboration

Browser

- Google Chrome

2. Hardware Requirements

Processor

Intel Core i5 / AMD Ryzen 5 or above

RAM

- Minimum: 8 GB
- Recommended: 16 GB

Storage

Minimum 5 GB free space

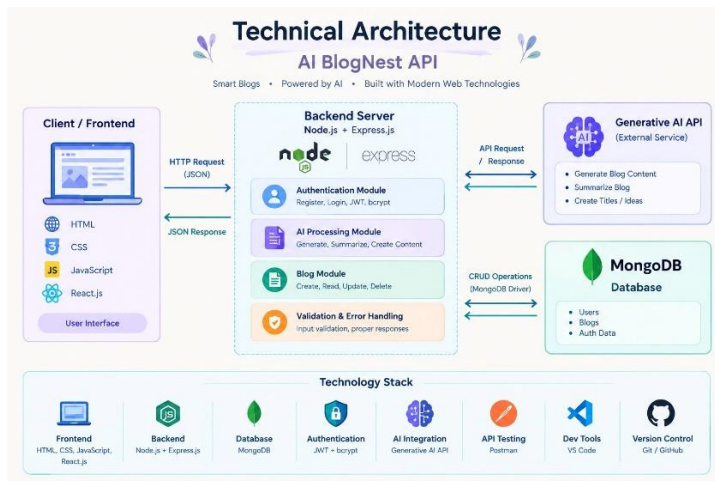
Internet Connection

Required for:

- AI API communication
- MongoDB Atlas connection
- Package installation

Project Architecture:

Technical Architecture



AI Blog Nest API follows a layered backend architecture.

Client / API Testing Layer

- Postman
- Frontend application (optional)

Backend Layer

- Node.js
- Express.js
- REST APIs
- JWT Authentication
- Middleware

Database Layer

- MongoDB
- Mongoose
- MongoDB Atlas

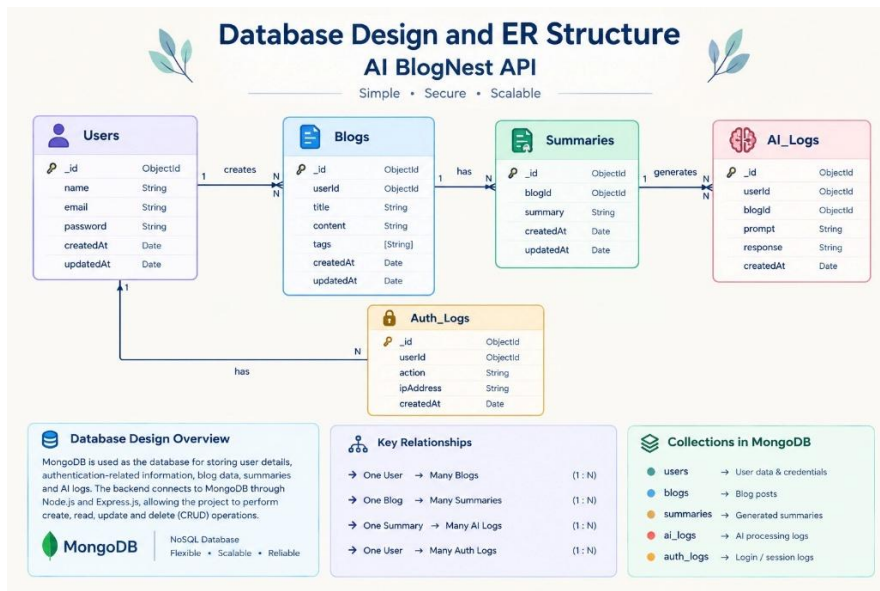
AI Layer

- AI/LLM API
- Prompt Engineering
- Content Generation
- Text Summarization

Security Layer

- JWT Authentication
- Password Hashing
- Protected Routes
- Input Validation
- Environment Variables

ER Diagram:



Collections:

Users

Stores registered user information.

Fields:

- _id
- name
- email
- password
- role
- createdAt

Blogs

Stores blog information created by users.

Fields:

- _id
- title
- content
- category
- author
- url
- createdAt
- updatedAt

AI Interactions

Stores AI-related requests and responses where required.

Fields:

- _id
- userId
- topic
- category
- prompt

- response
- type
- createdAt

ER Diagram Explanation

A **User** can create multiple blog posts. Each blog document contains information such as title, content, category, author, and timestamps.

Users can also interact with AI services to generate blog content or summarize existing content. AI-related requests and responses can be stored in the **AI Interactions** collection for tracking and future analysis.

The relationship between Users and Blogs represents the ownership of blog posts, while AI Interactions represent the AI operations performed by users.

Features

Core Backend Features

- **User Registration**
- **User Login**
- **JWT Authentication**
- **Protected Routes**
- **Middleware**
- **CRUD API**
- **File Upload**
- **Role-Based Access Control**

AI Features:

1. AI Blog Content Generation

The API generates blog content based on:

- Topic
- Category

Example:

```
{  
  "topic": "Node.js",  
  "category": "Web Development"  
}
```

The AI service processes the prompt and generates structured blog content.

2. AI Text Summarization

The summarization API accepts text and generates a shorter summary containing the important information.

Example use cases:

- Summarizing long blog posts
- Creating short descriptions
- Generating article previews

3. AI-Assisted Content Creation

AI can help users create initial blog drafts instead of manually writing the complete article.

4. Natural Language Processing

The AI service processes natural-language prompts and generates relevant responses.

Roles and Responsibilities:

Admin

The Admin can:

- Manage users
- View blogs
- Update or delete inappropriate content
- Monitor application activity
- Manage application resources

User

The User can:

- Register
- Login
- Create blogs
- View blogs
- Generate AI blog content
- Summarize content

User Flow

1. User Registration
2. User Login
3. Password Verification
4. JWT Token Generation
5. Token Sent with Protected Requests
6. Dashboard/API Access
7. Create or View Blogs
8. Submit AI Content Request
9. AI Service Processes Prompt
10. AI-Generated Content Returned

11. User Creates/Updates Blog
12. Blog Stored in MongoDB
13. User Can Retrieve or Manage Blog

MVC Pattern

The **AI Blog Nest API** follows the **MVC (Model-View-Controller)** architecture.

Model Layer

Responsible for database schemas and data structures.

Models include:

- User Model
- Blog Model
- AI Interaction Model

Mongoose is used to define and manage MongoDB schemas.

Controller Layer

Responsible for application business logic.

Controllers handle:

- Registration
- Login
- Authentication
- Blog creation
- AI content generation
- Text summarization

View Layer / Routes

Since this is a REST API backend, the View layer is represented through API responses and routes.

Authentication Routes

- Register
- Login
- Logout

Blog Routes

- Add Blog
- Get All Blogs
- Get Blog by ID
- Update Blog
- Delete Blog

AI Routes

- Generate Blog Content
- Summarize Content

Advantages of MVC

- Separation of Concerns
- Better Code Organization
- Easy Maintenance

- Reusable Components
- Easier Debugging
- Improved Scalability
- Easier Team Collaboration
- Better Testing

Project Setup and Configuration:

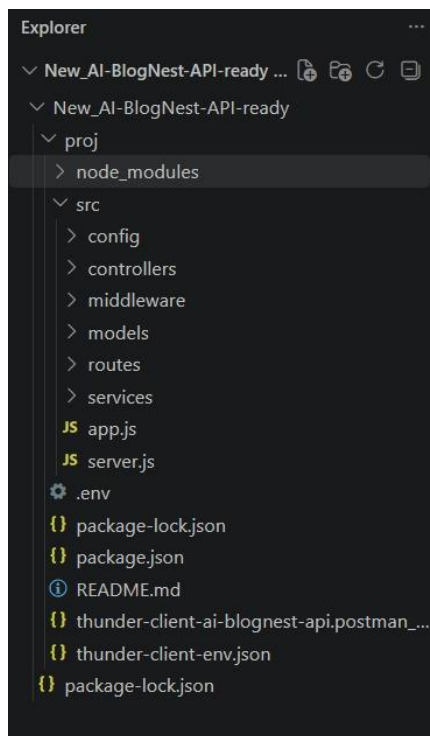
Project Structure

Create the project folder: AI-BlogNest-API

Inside the project: server/

Server Folder

Contains the backend application.



Server Setup

mkdir server

- npm start
- npm install
- dir
- dir -Directory
- cd proj
- dir
- npm install
- npm audit fix

- `npm install --save-dev nodemon`

`cd server`

`npm init -y`

Install Backend Packages:

`npm install express mongoose cors dotenv bcryptjs jsonwebtoken`

Install AI package:

- `npm install open ai`
- Additional packages:
- `npm install multer`
- Development dependency:
- `npm install nodemon --save-dev`

Backend Development

The backend contains the following major components:

Authentication APIs

Authentication APIs provide:

- User registration
- User login
- Password hashing
- JWT token generation
- Protected route access

JWT Implementation

After successful login, the server generates a JWT token.

The client sends the token in the Authorization header:

Authorization: Bearer <JWT_TOKEN>

The authentication middleware verifies the token before allowing access to protected APIs.

- **Blog APIs** - The Blog APIs implement complete CRUD operations.
- **Create Blog** - POST `/api/blog/addbook`
- **Fetch Blogs** - GET `/api/blog/getBooks`
- **AI API Integration** - AI Blog Nest integrates an AI service to provide intelligent content-processing capabilities.

- **AI Content Generation API** - Example endpoint: POST /api/ai/generate-content-blog

AI API Explanation:

Purpose

The AI APIs provide intelligent content-generation and summarization functionality to reduce manual writing effort and improve content-processing efficiency.

Request Structure

Requests are sent to the backend as JSON.

Example:

```
{  
  "topic": "Importance of Cybersecurity",  
  "category": "Cybersecurity"  
}
```

Response Structure

The backend returns a JSON response containing the status and AI-generated result.

Database Development:

MongoDB is used for persistent storage.

Database Collections

- Users
- Blogs
- AllInteractions

MongoDB Connection

The MongoDB connection is configured through an environment variable:

MONGO_URI=your_mongodb_connection_string

Mongoose establishes the database connection from the backend.

```
New_AI-BlogNest-API-ready > proj > .env  
  Import to Postman  
1  PORT=5000  
2  MONGO_URI=mongodb+srv://tejasarathy10_db_user:sai2007@cluster0.xkf15u5.mongodb.net/Blog_net?appName=Cluster0  
3  JWT_SECRET=change_this_to_a_long_random_secret_string  
4  GEMINI_API_KEY=AQ.Ab8RN6JUAWtr3iNC61ScKsDPC874MALHXaa0jUNW3MwDTRZaSg  
5  GEMINI_MODEL=gemini-2.5-flash  
6  Test-NetConnection localhost -Port 5000
```

Project Execution

Step 1: Open the Server Folder

```
cd server
```

Step 2: Install Dependencies

```
npm install
```

Step 3: Configure Environment Variables

- Create: .env
- Example: PORT=5000
- MONGO_URI=your_mongodb_connection_string
- JWT_SECRET=your_secret_key
- OPENAI_API_KEY=your_api_key

Step 4: Start the Backend

- npm start
- or, if Nodemon is configured:
- npm nodemon src/server.js

Backend URL

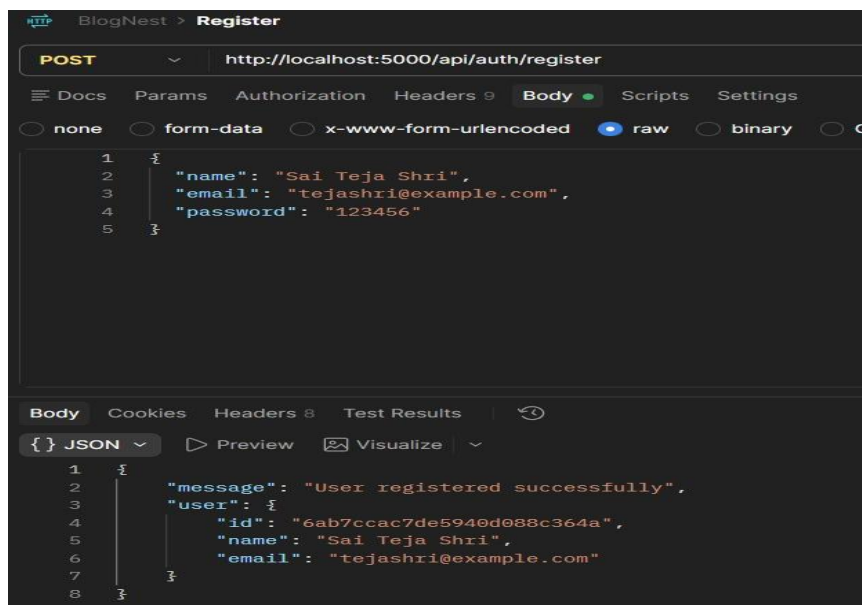
http://localhost:5000

API Testing Using Postman

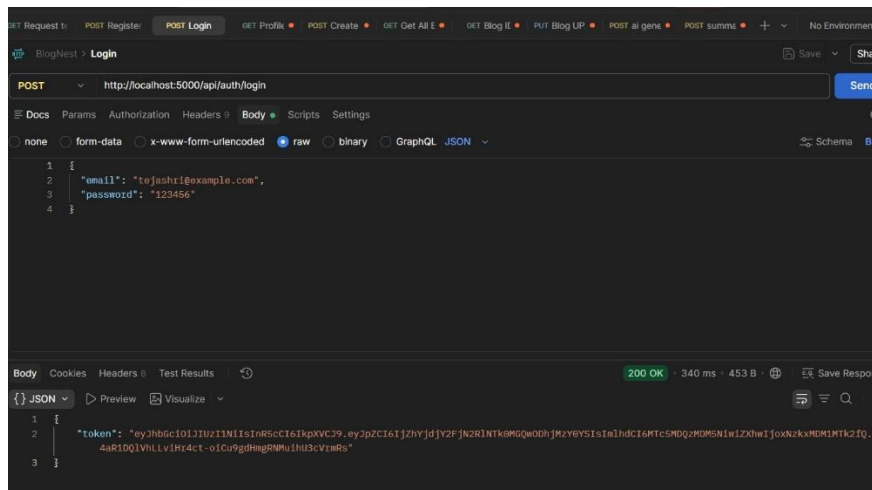
Postman is used to test all backend APIs.

Authentication Testing:

Register



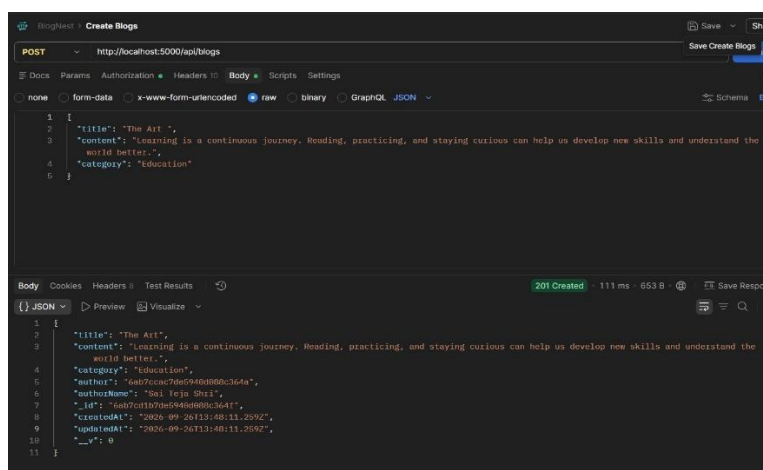
Login



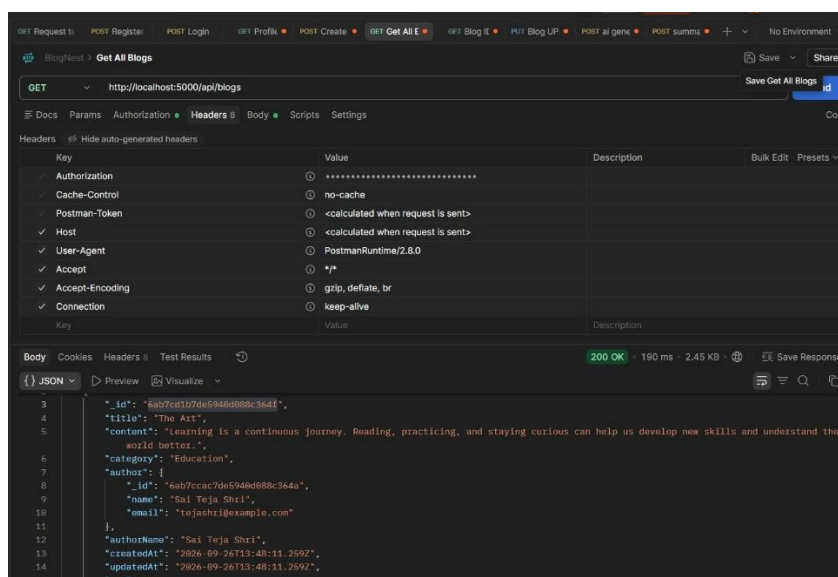
Blog API Testing

The following operations are tested through Postman:

- Create Blog

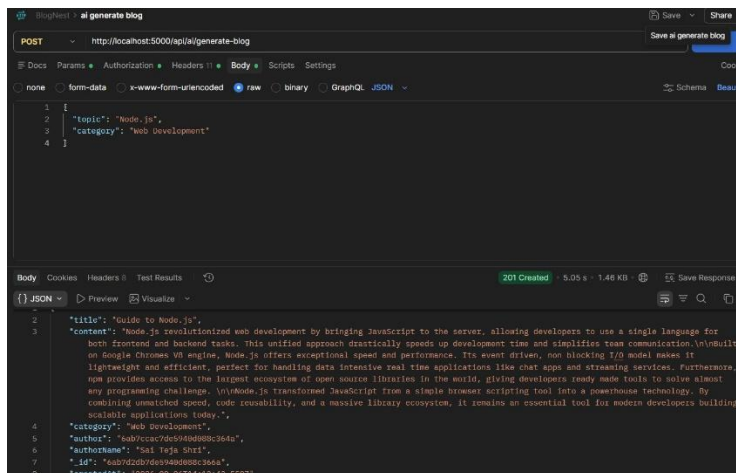


- Fetch Blogs

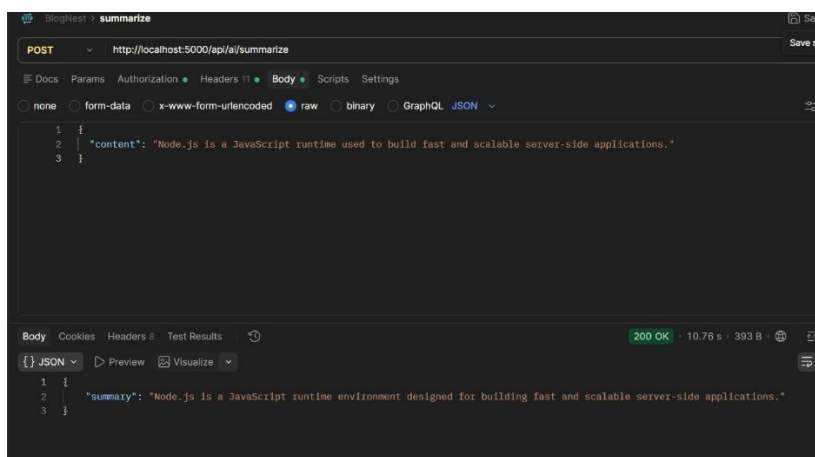


AI API Testing:

Generate Blog Content



Summarize Content



1. **Server Running** - Server running on port 5000
2. **MongoDB Connection** - Shows successful database connection.
3. **User Registration** - Postman screenshot showing registration request and response.
4. **User Login** - Shows successful login and JWT token response.
5. **Protected API** - Shows JWT authentication being used to access a protected endpoint.
6. **Blog Creation** - Shows the request and successful blog creation response.
7. **Blog Retrieval** - Shows all blogs returned from MongoDB.
8. **AI Content Generation** - Shows: POST `/api/ai/generate-content-blog` with topic and category and the AI-generated response.
9. **AI Summarization** - Shows the submitted text and generated summary.

10. Error Handling

Shows examples such as:

- Invalid token
- Missing required fields
- Invalid request
- Unauthorized access
- Resource not found

Demo Video Link: [Demo Video](#)

Source Code Drive Link: [GitHub](#) / [Google Drive](#)

Final Project Summary:

AI Blog Nest API demonstrates the integration of modern Artificial Intelligence capabilities with a secure RESTful backend architecture. The project combines **Node.js**, **Express.js**, **MongoDB**, **JWT authentication**, and **AI APIs** to provide secure blog management along with AI-powered content generation and summarization.

The application demonstrates how AI can be integrated into a conventional backend system to automate content-related tasks. The MVC architecture provides separation of responsibilities, while JWT authentication and protected routes improve application security. MongoDB provides persistent storage for users and blog information, and Postman enables systematic API testing.

Demo Video Link: [Demo Video](#)

Source Code Drive Link: [GitHub](#) / [Google Drive](#)